Project Task 03 - Submission Sheet

Software Testing - Graph Coverage

Names: [Kavya]
Group: [Group 4]

Software Testing - Summer 2024 - June 7, 2025 - Sibylle Schupp / Daniel Rashedi

Task 3 - Graph Coverage

a) Measure the coverage of your given project test suite (which includes the existing test suite as well as the tests that you created in previous project phases) by three graph coverage criteria which you can freely choose. Describe each individual result in 2-3 sentences.

Answer:

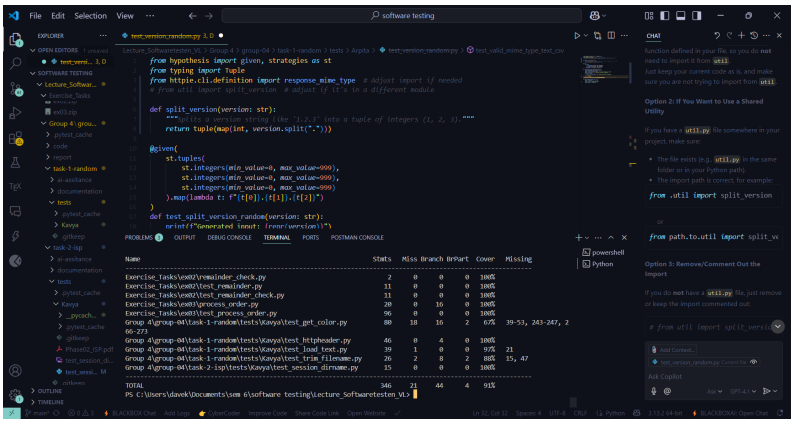

Figure 1: Screenshot of outcomes

Test Coverage Summary and Analysis

Fully Covered Files (100%)

- **remainder_check.py**: This simple module is completely tested. All logic paths are exercised with no missed lines or branches.
- **test_remainder.py** and **test_remainder_check.py**: These tests fully validate the functionality of `remainder_check`. The tests ensure all control flow paths are checked.
- **process_order.py**: All 20 statements and 16 branches are fully covered. The test suite includes diverse edge cases such as empty inputs, coupon variations, and discount logic.
- **test_process_order.py**: This test file provides complete validation of `process_order`, ensuring 100% statement and branch coverage.
- **test_session_dirname.py**: All test paths are fully exercised with no lines or branches missed.
- **test_httpheader.py**: Have all lines executed, showing good test thoroughness
- **test_session_dirname.py**: cover all 15 statements, no branch is missed and its 100% coverage

Partially Covered Files

- **test_trim_filename.py** (88%): Two missed lines (15 and 47) may reflect complex file name or trimming edge cases. Additional hypotheses or boundary value tests may be needed.
- **test_load_text.py** (97%): Only one line (39) remains uncovered. It likely represents a rare fallback or conditional branch not yet triggered.
- **test_get_color.py** (67%): This file has the most uncovered logic. Lines 39–53, 243–247, and 266–273 are not tested, suggesting missing test coverage for alternative branches, exceptions, or deeply nested logic.

Overall Coverage

The project achieves **91% overall coverage**, including 346 statements and 44 branches. While core logic is well-tested, several files can be improved by adding tests targeting edge cases and complex conditions.

---

b) Extend the test suite with own tests, which have to fullfil **one** of the following criteria:

1) Increase the coverage values of all three coverage criteria that you applied in the previous subtask with at least 10 tests *per group member* (compare and describe the effects on coverage for each individual test), OR
2) Reveal a new bug in the software project (describe the bug, its context, and a potential fix in detail)

Answer:

```
    test_trim_filename.py


def test_trim_filename_full_trim():
    filename = "abc.txt"
    max_len = 2  # Less than length of "abc"
    trimmed = trim_filename(filename, max_len)
    assert len(trimmed) <= max_len

def test_trim_filename_no_extension():
    filename = "abcdef"
    max_len = 3
    trimmed = trim_filename(filename, max_len)
    assert len(trimmed) <= max_len
    assert not trimmed.endswith('.txt')  # no extension to preserve

def test_trim_filename_zero_max_len():
    filename = "file.txt"
    max_len = 0
    trimmed = trim_filename(filename, max_len)
    assert trimmed == '' or len(trimmed) <= max_len
```

```
    test_load_text.py

def test_load_text_file_raises_oserror(tmp_path):
    # Create a file and remove it to trigger OSError when loading
```

```python
    file_path = tmp_path / "nonexistent.txt"
    arg = KeyValueArg(orig=str(file_path), value=str(file_path))
    with pytest.raises(ParseError):
        load_text_file(arg)
```

```python
    test_get_color.py

def test_get_color_with_missing_color():
    # Color not in palette
    assert get_color("not_a_color", "500") is None

def test_get_color_with_non_dict_color_value():
    # Use a palette with a direct string value instead of dict
    palette = {PieColor.GREEN: "#00FF00"}
    result = get_color(PieColor.GREEN, "any_shade", palette=palette)
    assert result == "#00FF00"
```

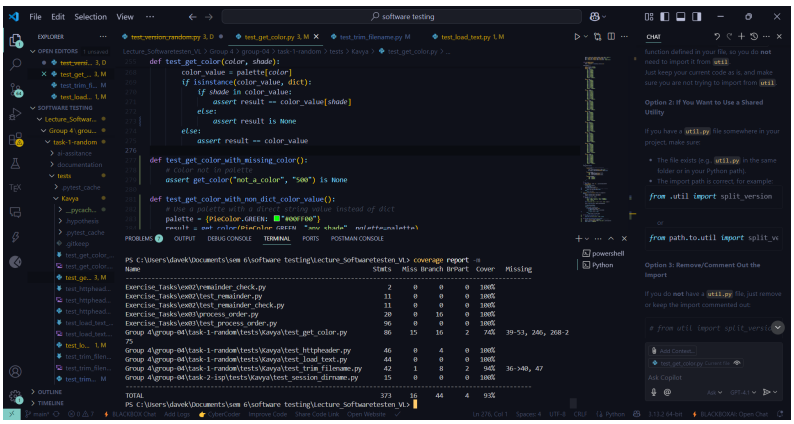

Figure 2: Screenshot of outcomes

AI Review Task

Manually conduct *SymPrompt* to let your LLM construct test cases. List the coverage results for the individual test cases. Add any code relevant for your analysis to the answer:

Answer:

[Group: Your LLM]
  [Group: Your Steps]
  [Group: Your Results]
  ...